Grenze International Journal of Engineering and Technology, January Issue

Identification of Indian Languages using Naïve Bayes, CNN, LSTM, and HMM

Harpreet singh[1], Rachait Talwar[2], Enjula Uchoi[3] and Avnish Thakur[4]
[1-4]Lovely Professional University, Phagwara, India
Email: harpreet162004@gmail.com, rachaittalwar159, enjulapaintoma, avi.himachal}@gmail.com

***Abstract*—This paper investigates the creation of a machine learning system for language identification, focusing on four Indian languages: Malayalam, Hindi, Tamil, and Kannada. We applied several models, including Gaussian Naïve Bayes (NB), Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Hidden Markov Models (HMM), to effectively classify languages based on text inputs. Our research used a comprehensive multilingual dataset, and the evaluation metric was primarily accuracy. The experimental results revealed that both the HMM and Naive Bayes models achieved impressive accuracy rates exceeding 99%, while CNN and LSTM models also delivered competitive outcomes. These results demonstrate the effectiveness of machine learning models for language classification and highlight the potential of these models in advancing natural language processing (NLP) for Indian languages. Our findings contribute to the growing body of work in multilingual NLP, specifically for underrepresented Indian languages, and offer insights for further enhancements in automatic language recognition systems.**

***Index Terms*—Naive Bayes, CNN, LSTM, HMM, Language Classification, Natural Language Processing.**

I. INTRODUCTION

India is a land of 22 officially recognized languages and at least over 1,600 dialects. Only 4 — Malayalam, Hindi and two other Dravidian languages (Tamil and Kanada). Because of the diversity in the languages spoken worldwide, applications such as speech recognition and text processing or machine translation must be able to support multiple languages.

Given this diversity, automatic language identification systems are vital for developing applications such as speech recognition, text processing, and machine translation that can adapt to multilingual settings. Identification of Language is the problem of determining which language a given text or speech utterance is in, without knowing how it was written, said or vocalized.

This process is crucial in the preprocessing stages of various NLP applications. While language identification is relatively simple when comparing languages from different families (e.g., English vs. Mandarin), it becomes significantly more challenging when working with languages that share similarities in structure, vocabulary, or script.

We discussed the different models—Gaussian Naive Bayes, CNN, LSTM and HMM—in this paper that we used to recognize Indian languages. By testing the performance of those models, we will be able to elaborate on which methods prove most effective for this sort of classification task — and where are areas to improve.

Grenze ID: 01.GIJET.11.1.224

II. Problem Statement

The identification of Indian languages is a crucial task in natural language processing (NLP) due to the linguistic diversity in India. This research focuses on developing and comparing multiple machine learning models—Naive Bayes (NB), Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Hidden Markov Models (HMM) for the identification of four Indian languages: Malayalam, Tamil, Kannada, and Hindi. The study aims to evaluate the accuracy and efficiency of these models in classifying short text samples from different Indian languages. The objective is to provide insights into the performance of traditional and deep learning methods for language identification, enabling more effective text processing and language detection in multilingual settings.

III. Literature Review

Language identification has been widely researched, especially for Western languages like English, Spanish, and French. In contrast, fewer studies have focused on Indian languages, despite the significant linguistic diversity in India. Historically, language identification methods were based on rule-based systems or statistical approaches like n-grams and character frequency analysis. However, these methods struggle with text that is phonetically similar or uses similar character sets, as is often the case with Indian languages. Recent advances in machine learning and deep learning, language have improved immensely in efficiency Ambikairajah et al. [10] discusses identification systems. Share a comprehensive tutorial on language identification. Methods that determine the promise of neural networks in this field. Their work shows some benefits of using deep learning models, learn complex features from Information, therefore, increases the rate of identification. Additionally, models such as LSTM have shown promise for language classification tasks due to their ability to capture temporal dependencies in sequential data. Zhan et al. [12] introduced an improved LSTM model specifically designed for language identification, which achieved remarkable results on various datasets. This aligns with findings by Graves et al. [3], who demonstrated the effectiveness of deep recurrent neural networks for speech recognition tasks. Furthermore, the utilization of CNNs for end-to-end language identification has gained widespread attention. Wang et al. [15] showcased how CNNs can effectively process audio features to identify languages, paving the way for similar approaches in text-based identification. This shift towards deep learning models reflects the growing trend of leveraging neural network architectures for more robust language identification systems, moving beyond traditional methods. Lecun et al. [1] further underscore the transformative impact of deep learning, noting its ability to automate feature extraction and learn complex patterns in data, which is particularly beneficial for language identification tasks.

Thus, noteworthy work exists on language identification for Western languages, but the same is relatively unexplored for Indian languages. This is a field where more research and development into the technology can be undertaken for multilingual natural language processing.

A. Naive Bayes

Naive Bayes is arguably the simplest and fastest algorithm for text classification tasks. Bayes' Theorem has it that this measures the probability of a given file being associated with a specific category (or language) determined by term occurrence rate within the manuscript. Notwithstanding its assumption of independence among features (words), Naive Bayes is quite successful in linguistics Classification is derived from the difference-making attributes inherent in text data, as vindicated by the seminal works of Joachims et al [7]. Two distinct forms of Naive Bayes that frequently used are: Gaussian and Multinomial. The Gaussian variant assumes that the characteristics are continuously distributed, and this is rather not appropriate for text classification tasks. On the contrary, the Multinomial It is adapted especially for text classification where word frequencies are discrete distributions. This version is peculiarly useful in many sorts of research, such as those by Zhan et al. [12], proving its applicability to language identification tasks. In application, the Multinomial Naive Bayes classifier efficiently manages the elevated dimensionality inherent in text data by taking into account the occurrence frequency of individual words within the document, thereby enabling it to produce well-calibrated forecasts even under low computational power. Simplicity of the algorithm Make it a more preferred choice for preliminary investigations in linguistic studies Classification, especially for datasets of different languages. Indian languages, in fact, where linguistic diversity poses distinct challenges.

B. CNNs for Text Classifications

While Convolutional Neural Networks (CNNs) are widely used for image classification, their application to text categorization has proved challenging because these models operate on ordered sequences of tokens. CNNs uses

filters that captures local n-grams (a few, nearby words) to higher-level abstract features. In the context of NLP, using CNNs allows not only to capture syntactic but also semantic structures in a sentence. Since CNNs are good at exploiting spatial relationships among words and these texts may be shorter, which hinders a neural network in understanding the text as sequential data needed for an RNN-based approach to work well — this architecture should perform better than an exposition one in language identification.

C. LSTM Networks

Long Short-Term Memory (LSTM) networks are a type of Recurrent Neural Network (RNN) designed to handle sequential data. LSTMs can remember past information and use it to predict future elements in a sequence, making them ideal for tasks involving natural language processing. LSTMs have been applied to language modeling, translation, and sequence prediction tasks. In this study, LSTMs are used to capture the sequential nature of text and handle the dependencies between words, enabling more accurate language prediction.

D. Hidden Markov Models (HMMs)

Hidden Markov Models are statistical structures that hypothesize the existence of an underlying mechanism like a string of words, which causes the observable sequence, like the character count in the text. HMMs have applications in a wide range of areas; in speech recognition, they serve as powerful instruments for different tasks. The areas like text-based language identification are characterized probabilistically by these models for sequences. In this research work, we have used the HMMs to outline the progression count of characters in a text and makes associations with certain languages classes.

IV. METHODOLOGY

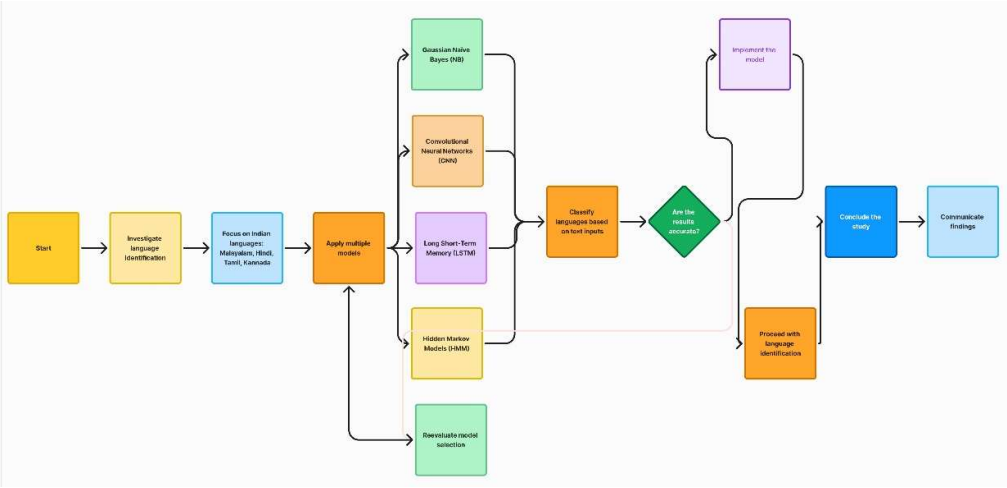

Figure 1. Language Identification Framework

A. Dataset

This paper uses a dataset prepared by gathering four different Indian languages, that is Malayalam, Hindi, Tamil, and Kannada. They present diversified linguistic structure. Before performing an analysis on it, preprocessing is done on the dataset for removing irrelevant objects, normalizing the punctuation marks, and splitting the text into words or characters based upon the requirement of the input of the model. There are thousands of text entries for the dataset of each language, so that there will be a rich mix of sentence structures and vocabularies, which prevents overfitting in the system. The label identifying the language is assigned to each text entry, so it is well-suited applications in supervised learning.

B. Data Preprocessing

Tokenization: It is basically splitting the text into tokens. This step is necessary in processing raw text into some order. It is one of the important procedures in NLP through which a text can be broken down into small elements called tokens for machine processing capabilities.

Vectorization: The text data is converted into a numerical format. For the Naive Bayes models, we employ Term Frequency-inverse document frequency (TF-IDF) vectorization, which takes into account both the frequency of a

term in a document and its mutual frequency across the entire corpus. This enhances linguistic discrimination by overemphasizing relatively less significant factors terminology, as seen in earlier studies performed by Joachims [7]. For the CNNs and LSTMs, we used the word embeddings that transform words into dense vectors that capture their semantics relationships, a methodology proposed by Mikolov et al. [2].
Padding: In deep learning models such as CNNs, LSTMs, we insist the sequences be the same length. We add zeros to make the text sequences of uniform length to more compact sequences. This uniformity leads to
effective batch processing in deep learning frameworks promoting better training relationships.

C. Feature Extraction

Naive Bayes: For the Naive Bayes models, features are then extracted using TF-IDF vectorization. This technique the importance of vocabulary in a text, assigning much stronger focus on infrequently used vocabulary, which can dramatically help in the language discrimination, as emphasized by Ambikairajah et.al. [10].
CNN, LSTM: In the deep learning models-an embedding a layer is applied to convert every single word in the text sequence into compact representation in terms of real-valued vectors, capturing efficiently its semantic meaning. CNNs extract n-gram features, but LSTMs are designed to learn long-term dependencies in the sequence. The efficiency of CNNs to Language identification has been put forward by Wang et al. [15], and capabilities of LSTM when handling sequential data were first invented by Hochreiter and Schmidhuber [4].
HMM: In the case of HMM, it replaces every character insert a number sequence into the text. These sequences are
to train the HMM model on the sequential structure of each language, predictive probability for a particular character sequence. This technique aligns with earlier research works on language identification using HMMs, which demonstrate their utility of explaining the complexities identified within linguistic patterns.

V. Model Architecture

A. Naive Bayes

Both Gaussian and Multinomial Naive Bayes models were used. The Gaussian model is suited for continuous input (word frequencies as a continuous distribution), while the Multinomial model handles discrete word counts.

$$P(C_k|X) = \frac{P(C_k)\prod_{i=1}^{n} P(x_i|C_k)}{P(X)}$$

B. CNN

The CNN model consists of an embedding layer followed by a series of convolutional and pooling layers to extract higher-level features from the text. A final dense layer with softmax activation predicts the language class. Cross-entropy loss is often used as an objective function in multiclass classification tasks.
It is defined as:

$$L = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$$

C. LSTM

The LSTM model comprises an embedding layer, a single LSTM layer to capture the sequence dependencies, and a fully connected output layer for classification. The model also includes dropout layers to prevent overfitting. The LSTM calculates a hidden state (ht) at time t using its input and previous hidden state.
The mathematical representation is as follows:

$$h_t = \sigma(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

D. HMM

The training of the HMM model is done on character sequences of all languages. It applies Gaussian Hidden Markov model for modeling and predict the most probable language for a given sequence.

In an HMM, the probability of observing a sequence O = (o1, o2,…, oT) given hidden states Q = (q1, q2,…, qT) is calculated as:

$$P(O \mid Q) = P(q_1) \prod_{t=2}^{T} P(q_t \mid q_{t-1}) \prod_{t=1}^{T} P(o_t \mid q_t)$$

Where $P(q_t \mid q_t\text{-}1)$ denotes the transition between hidden states and P(ot | qt) is the emission probability of observation to for given state qt.

VI. Results and Discussion

All the models were tested on the same test set with accuracy is the most frequently applied measure. Summary of the results as in:

TABLE I. MODEL PERFORMANCE SCORE ACCURACY

Model	**Accuracy (%)**
Gaussian Naive Bayes	99.33
Multinomial Naive Bayes	99.67
CNN	99.66
LSTM	95.32
HMM	99.67

These results indicate that while traditional models like Naive Bayes and HMM offer high accuracy, deep learning models such as CNN and LSTM still provide reliable performance, though slightly lower in comparison. The findings highlight the effectiveness of both classical and advanced machine learning techniques for language classification tasks.

Confusion Matrix (CNN)

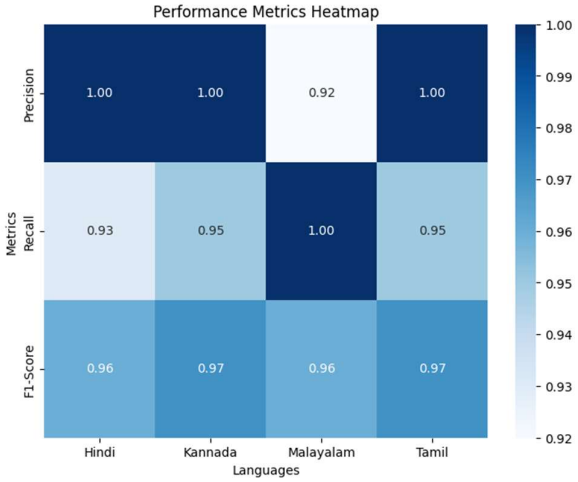

Figure 2. Language Identification Confusion Matrix Score

TABLE II. LANGUAGE IDENTIFICATION RESULT

	Precision	**Recall**	**f1-score**	**Support**
Hindi	1.00	0.93	0.96	14
Kannada	1.00	0.95	0.97	79
Malayalam	0.92	1.00	0.96	113
Tamil	1.00	0.95	0.97	93
Accuracy			0.97	299
Macro avg	0.98	0.96	0.97	299
Weighted avg	0.97	0.97	0.97	299

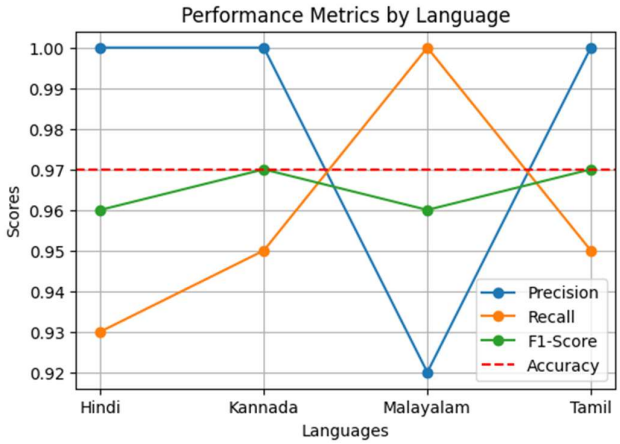

Figure 3. Line graphs of Language Identification

The results are that the Multinomial Naïve Bayes and HMM models had the highest accuracy, sometimes suggesting, for example, more simple models that got better results than heavy deep learning models with well-formatted datasets and classification-oriented task types. This means that, under some circumstances, the conventional methods may deliver satisfactory performance with the computation omitted overhead of more elaborate models. In contrast, the CNN and LSTM models achieved a competitive accuracy. These models suggest good promise for future tasks that involve longer or more elaborate forms of writing because they are designed to detect intricate patterns and relationships between the data. Their ability to learn across larger context windows renders them particularly useful for applications involving a deeper understanding of linguistic nuances. Overall, this
study emphasizes the role of model selection based concerning the specific properties of the dataset as well as inherent properties that is going to be helpful in such classification tasks.

F. Error Analysis

The errors while prediction occurred due to short or vague sentences or expressions in several languages. For example, sentences containing loanwords or code-mixed elements—a common feature of multilingual milieus-posed a more difficult task. Future improvements might include incorporating a character-based model to better handle these pre-trained multilingual embeddings have been widely used to boost robustness.

VII. CONCLUSION

This paper discusses the development of a language identification system based on the combination of machine learning and deep learning techniques, including Naive Bayes, CNN LSTM, and HMM models. Although there are some differences in performance among the models, each one contributes to
It lays the ground for further, advanced language Identification Systems.
Looking ahead, our future work will aim to refine these models by integrating more sophisticated techniques, such as transformer-based architectures, which have shown great promise in various natural language processing tasks. We will also focus on enhancing the system's ability to handle ambiguous text more effectively and expanding its support to encompass a wider array of Indian languages. By doing so, we hope to improve the system's robustness and make it more applicable in real-world multilingual environments, ultimately fostering better communication and understanding across diverse linguistic communities.

REFERENCES

[1] Y. Lecun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436-444, May 2015.
[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in Proceedings of the International Conference on Learning Representations (ICLR), 2013.
[3] A. Graves, A. Mohamed, and G. Hinton, "Speech Recognition with Deep Recurrent Neural Networks," in Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2013, pp. 6645–6649.
[4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[5] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.
[6] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, 2018.
[7] T. Joachims, “Text categorization with Support Vector Machines: Learning with many relevant features,” in Proceedings of the European Conference on Machine Learning (ECML), 1998, pp. 137–142.
[8] I. Lopez-Moreno, J. Gonzalez-Dominguez, O. Plchot, D. Martinez, J. Gonzalez-Rodriguez, and P. Moreno, “Automatic language identification using deep neural networks,” 2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Florence, Italy, 2014, pp. 5337-5341, doi: 10.1109/ICASSP.2014.6854622.
[9] E. Ambikairajah, H. Li, L. Wang, B. Yin, and V. Sethu, “Language Identification: A Tutorial,” in *IEEE Circuits and Systems Magazine*, vol. 11, no. 2, pp. 82-108, Second quarter 2011, doi: 10.1109/MCAS.2011.941081.
[10] R. Cole, J. Inouye, Y. Muthusamy, and M. Gopalakrishnan, “Language identification with neural networks: a feasibility study,” in Communications, Computers and Signal Processing, 1989. Conference Proceeding., IEEE Pacific Rim Conference on, 1989, pp. 525–529.
[11] Q. Zhan, L. Zhang, H. Deng, and X. Xie, “An Improved LSTM For Language Identification,” 2018 14th IEEE International Conference on Signal Processing (ICSP), Beijing, China, 2018, pp. 609-612, doi: 10.1109/ICSP.2018.8652445.
[12] Y.K. Muthusamy, R.A. Cole, and B.T. Oshika, “The OGI multi-language telephone speech corpus,” in Proc. ICSLP ’92, vol. 2, October 1992, pp. 895-898.
[13] A. Stolcke, “SRILM - An Extensible Language Modeling Toolkit,” in *Proc. ICSLP, September 2002.
[14] Y. Wang, H. Zhou, Z. Wang, J. Wang, and H. Wang, “CNN-Based End-To-End Language Identification,” 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Chengdu, China, 2019, pp. 2475-2479, doi: 10.1109/ITNEC.2019.8729388.